2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00117

EIGEN: Enabling Efficient 3DIC Interconnect with Heterogeneous Dual-Layer Network-on-Active-Interposer

Siyao Jia*, Bo Jiao*, Haozhe Zhu†, Chixiao Chen†, Qi Liu, Ming Liu
State Key Laboratory of Integrated Chips and Systems
Frontier Institute of Chip and System, Fudan University
{syjia23, bjiao22}@m.fudan.edu.cn, {zhuhz, cxchen, qi_liu, liuming}@fudan.edu.cn

***Abstract*—Chiplet-based 3DICs have emerged as modular solutions for large-scale, high-performance computing systems. However, unlike monolithic Network-on-Chips (NoCs), 3DICs using active interposers encounter difficulties in managing heterogeneous and high-load network traffic patterns. The emerging field of Networks-on-Active-Interposer (NOAI), independently designed from top dies' interconnects, is desired to address these challenges by supporting flexible topologies, low-latency memory access, and reduced traffic congestion.**

To satisfy the requirements, we propose a heterogeneous dual-layer interconnect architecture, EIGEN, for chiplet-interposer systems, along with a reinforcement learning (RL)-based routing framework, which can provide efficient and flexible communication for chiplet-based 3DICs. EIGEN features an application-aware switch-programable interconnection layer (AspLayer) and a dynamic packet-routing interconnection layer (DynLayer) on the interposer, respectively. To optimize inter-chiplet data communication, we also develop an RL-based path routing framework tailored to this dual-layer architecture. The RL framework's state space includes topology, network, and memory metrics, while the RL reward is set as the product of average packet latency and link utilization. The effectiveness of EIGEN is demonstrated through different applications where CPU, GPU, and AI accelerator chiplets are integrated via NOAI. Simulation results show that the proposed EIGEN achieves up to 67.16% latency reduction, 53.89% hops reduction and 11.21% runtime reduction compared to the state-of-the-art (SOTA) chiplets interconnect architectures. Furthermore, sensitivity analysis of network scaling shows that the EIGEN architecture and framework exhibit strong scalability, with latency reduction of 20.96% to 52.77% and hop reduction of 19.72% to 43.41% as the system scales from 4×4 to 32×32 chiplets.

***Index Terms*—Interconnection Network, Heterogenous Chiplet Integration, Reinforcement Learning, Heterogenous Architecture, Programmable Topology.**

I. Introduction

The evolution of modern computing towards chiplet-based architectures [1]–[3] addresses the rising costs and complexities of SoC manufacturing [4], [5]. By partitioning a virtually large monolithic SoC into smaller, modular chiplets, this approach not only enhances overall system yield [6], [7] but also enables rapid, versatile integration by reusing previously developed function dies. With the advances in 3D integration technologies such as through-silicon vias (TSVs) and hybrid bonding, active-interposer-based 3DICs are emerging as a more advanced architecture [8], [9]. Compared to 2.5D integration, 3DICs offer higher integration density and improved vertical memory access bandwidth, enabling better power efficiency and faster inter-chiplet data transfer. Fig. 1 illustrates a typical 3DIC [7], [10]–[12], where both active components, such as power management, circuit switches, and global memory buffers, as well as passive devices, such as routing metals and capacitors, are embedded in the base active interposer. One key difference of active interposers is the ability to implement the entire interconnect networks.

*The authors contributed equally to this work.
†Corresponding author: Chixiao Chen, Haozhe Zhu.

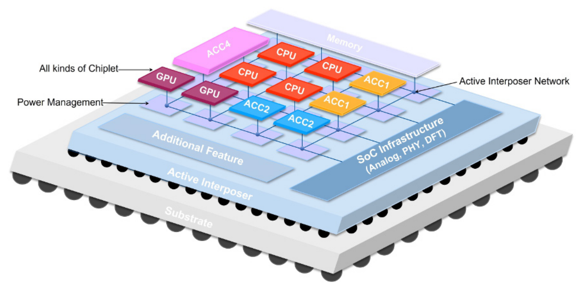

Fig. 1: An example of multi-chiplet 3DIC. Top dies are heterogeneous chiplets, such as CPUs, GPUs, and accelerators *et. al*. The base die is an active interposer, providing interconnection networks, memories, and external interfaces.

Despite these advancements, integrating interposers with chiplets introduces communication bottlenecks, undermining the benefits of chiplet technology. Bandwidth limitations stem from finite I/O pins and shared bandwidth by multiple components [13]. Moreover, NOAI struggles to address the varying bandwidth and latency requirements of different chiplets (e.g., CPUs, GPUs), particularly under multi-hop transmissions that increase latency and congestion. This creates new challenges for managing high-load and diverse traffic in NOAI.

Numerous studies have explored varied architectures using active interposers to boost interconnect performance and manage traffic. Asymmetric NoC distributions across chiplets and interposer [14] enhance efficiency, and IntAct introduces the first fabricated CMOS active interposer for distributed interconnections [11]. Other innovations include non-aligned interposer NoC [1] and Kite's series of chiplet interconnect

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00117

topologies [15], focusing on traffic optimization. GIA offers a reusable interposer supporting various configurations [16]. Despite these advancements, these methods face limitations in handling heterogeneous and high-load traffic, such as IntAct's narrow applicability and the non-aligned topology's limited versatility for different traffic types. GIA, while enabling a reusable interposer, struggles with diverse traffic. Additionally, various interconnection architectural strategies incorporating long express links [17] and multiplexers (MUX) integration [16], [18] aim to mitigate latency and congestion but increase complexity and power consumption. Dynamic topology adjustments [19] enhance specific applications but can introduce latency issues and instability in diverse systems. Previous static topology customizations [20]–[22], while effective in monolithic SoCs, fall short in chiplet-based systems.

To address the challenge of bandwidth and latency constraints introduced by active-interposer-based 3DIC under complex, high-load traffic, we propose EIGEN, a heterogeneous dual-layer interconnect architecture. EIGEN combines customizable interconnect topologies with interposer NoC for efficiency and flexibility. We also develop an RL-based routing framework to optimize routing based on EIGEN.

This paper makes the following contributions:

(1) **Dual-Layer Interconnect Architecture *EIGEN*:** We propose EIGEN, a heterogeneous dual-layer interconnect architecture for chiplet-interposer systems. EIGEN comprises two distinct layers with specific functional roles: the application-aware switch-programmable interconnection layer **(AspLayer)** optimized for efficient, high-throughput communication tailored to application demands, and the dynamic packet-routing interconnection layer **(DynLayer)** that manages auxiliary traffic and memory access. To the best of our knowledge, this work is the first to propose a dual-layer interconnect architecture combining customizable topologies with flexible NoC on the interposer, broadening the optimization search space for chiplet-based systems.

(2) **Reinforcement Learning-based Dual-layer Routing Framework:** Building on the EIGEN architecture, we propose an RL-based routing framework for EIGEN. This framework employs automated path selection strategies to choose the most appropriate paths for applications and customized topologies, aiming to maximize performance and energy efficiency.

(3) We evaluate the proposed EIGEN architecture using CPU, GPU, and AI benchmarks. Simulation results demonstrate performance improvements compared to SOTA architectures, with EIGEN achieving up to 67.16% latency reduction, 53.89% hops reduction, and 11.21% runtime reduction. This evaluation also validates the efficacy and robustness of EIGEN architectures in diverse scenarios.

II. Background and Motivation

A. Chiplet Integration with Interposer

In the post-Moore era, chiplet integration offers a new solution, allowing for systems with over 100 billion transistors and high yields [23], [24]. Composing monolithic SoCs into smaller, specialized chiplets on a silicon interposer enhances design and manufacturing efficiency [6], [7]. Proven by Intel Agilex [25] and BR100 GPGPU [26], chiplet technology reduces costs, supports heterogeneous integration, enabling modular hardware design [16].

Silicon interposers serve as the physical base die for chiplet integration and interconnection, using advanced packaging technologies like Cu-interconnect, microbumps (μbumps), and TSVs for efficient data and power transfer [27], [28]. As shown in Fig. 2, there are two types: passive and active interposers. Passive interposers, composed solely of metal layers without CMOS transistors, offer a cost-effective solution with simple point-to-point connections but cannot integrate routers or use repeaters for enhancing long-distance transmission speeds, necessitating internal routers within chiplets. Network links travel from the output channel through bumps to the interposer, traverse a long, unbuffered path, and then pass through another bumps to the router's input channel. This design necessitates die-to-die NoC connections for all chiplets, often adding overhead for electrostatic discharge (ESD) protection [29].

Active interposers [1], [11], [14], [30], shown in Fig. 1, integrate CMOS-based logic components, such as power management, clock generation, and I/O, directly on the interposers, enabling additional functionalities. Notably, they feature interconnect networks as seen in Fig. 2(c), allowing routers to be relocated to interposers, reducing top dies area [7]. Active interposers simplify communication by adding a single high-bandwidth hop from chiplets to interposer router, enabling flit movement between routers without die-to-die PHYs. They also support high-speed repeaters, changing the delay-distance relationship from quadratic ($O(l^2)$) to linear ($O(l)$), thus enhancing operational frequency and transmission range.

B. Interconnection Network on Chiplet-Interposer System

Monolithic SoCs typically use NoC for inter-component communication [31]. With chiplet technology, interposers enable connections between stacked chiplets. Unlike 2D designs, interposer-based systems face unique challenges. Due to chiplet technology fragmenting the interconnect network, communications between chiplets need to take additional hops through the interposer, which often employs older technology nodes and thus delivers lower performance compared to the chiplets. System performance is affected by component latency and network limitations, necessitating careful interconnection network design to address these challenges in chiplet-based systems. As analyzed above, active interposers alleviate some of these challenges by reducing the required bumps number and allowing routers to be relocated to the interposer, enhancing placement flexibility, also coupled with the use of repeaters to improve interconnect speeds. The advantages of active interposers open opportunities for exploring diverse network architectures to improve performance.

A non-symmetric interconnect network [14] was proposed to distribute NoC across multicore chips and the interposer, creating direct and indirect sub-networks that utilize interposer resources. While innovative, it primarily targets monolithic SoCs and overlooks chiplet decomposition, limiting its current

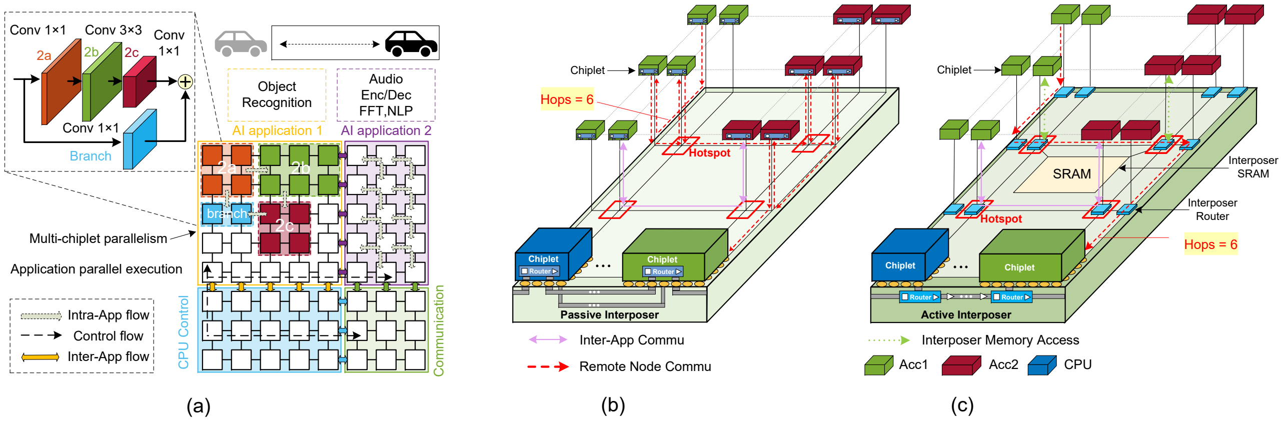

Fig. 2: Current application demands and challenges of chiplet-based systems. (a) The various parallel data flows and application requirements in chiplet-based systems. (b) 2.5D chiplet-interposer system with unbalanced communication and congestion problems. (c) 3D chiplet-interposer system. Communication bottleneck still exists with high latency.

relevance. IntAct [11] features distributed chiplet interconnection with four levels of interconnection but is tailored to specific chiplet characteristics, limiting its scalability. The non-aligned interposer NoC topology [1] alleviated high-traffic bottlenecks by balancing traffic through simultaneous interposer router links, yet it is suitable only for specific topologies and poses implementation challenges. Kite [15] offered a series of chiplet interconnect topologies that decouple on-chip and interposer networks, aiding in trade-off analysis, but its complex topologies are challenging to implement and require application-specific customization. GIA [16] developed a reusable universal interposer architecture with modified router microarchitectures featuring bypass links and MUX, supporting diverse chiplet assemblies but potentially causing link conflicts and longer routing.

These approaches modify network topology and routing paths to tackle interconnection challenges in chiplet-based systems, aiming to optimize traffic management. However, these solutions focus on specific issues, struggling to balance network versatility and efficiency. Consequently, they face limitations in effectively supporting diverse heterogeneous and high-load traffic patterns.

C. Communication Bottleneck in Chiplet-based System

Heterogeneous integration in chiplet-based systems, as illustrated in Fig. 1 and Fig. 2(a), leads to varying data requirements and traffic patterns among different chiplets. These systems are designed to enhance computational performance and power efficiency in response to the slowing pace of Moore's Law, accommodating both data-intensive and compute-intensive applications [32]. For example, concurrent execution of diverse applications such as object recognition and natural language processing tasks, each with unique data flow characteristics, is common [33]. This diversity in application execution results in varied and complex communication patterns among chiplets, emphasizing the need for adaptable interconnect strategies.

Fixed-topology NoC architectures, such as mesh, cmesh, and ring, offer sufficient flexibility for node-to-node communication via multi-hops. However, as network size and traffic volume grow, latency for long-distance communication between distant nodes becomes a significant bottleneck. As shown in Fig. 2(b) and Fig. 2(c), representing 2.5D and 3D systems, respectively, the hop counts required for such communications far exceed those for neighboring nodes, and the associated costs escalate in scenarios involving complex and high-load traffic.

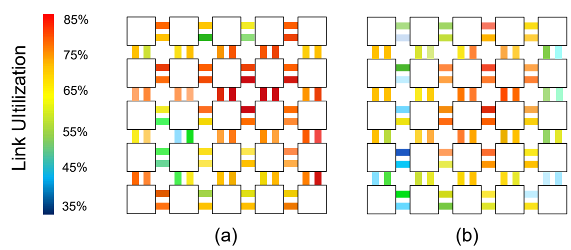

Fig. 3: The statistical results of network link utilization in chiplet-based systems. (a) 2.5D chiplet-based system. (b) 3D chiplet-based system.

We profiled and simulated interconnect network traffic for NoC-based 2.5D and 3D systems across various patterns, including uniform, neighbor, and shuffle. Our observations revealed a sharp deterioration in network conditions under dense traffic scenarios, particularly for paths involving multiple hops. As shown in Fig. 3, 2.5D and 3D chiplet-based systems experience this issue, with link utilization in the network center surpassing 80%, significantly higher than the 40% at border routers. Although active interposer interconnections and interposer memory help mitigate this congestion, inter-chiplet NoC still encounters substantial congestion challenges.

Router microarchitecture, which includes complex control logic and multiple pipeline stages, significantly contributes to latency and power consumption within interconnect networks. Based on research [34], routers account for over 80% of the network's total power consumption. Previous studies on programmable NoCs in monolithic many-core or 2.5D chiplet-based systems, such as SMART [18] and GIA [16], have introduced bypass links to enable programmable paths. AdaptNoC [19] further incorporates MUX and more links for dynamic topology changes. Despite these advancements, the network topologies remain flat and depend heavily on router-based forwarding, which may result in long queue latency and hotspot issues, particularly under high-load and complex traffic conditions.

To overcome the challenges posed by chiplet-interposer systems, it is crucial to develop new interconnection strategies that address bandwidth and latency constraints and the specific traffic characteristics of NOAI. These strategies must balance flexibility and efficiency, enabling chiplet-based systems to manage complex, data-intensive applications effectively.

III. Dual-Layer Interconnect Architecture

A. System Architecture Overview

Building upon prior discussions and established groundwork, we recognize the bottlenecks in NoC systems due to heterogeneous chiplet integration. These systems face critical issues like congestion and inefficient long-distance communication, exacerbated under complex and high-load traffic patterns inherent in modern computing systems. The necessity for an optimized interconnect network has become more apparent.

The key insights gained from our analysis led to the conceptualization of the EIGEN architecture. This model proposes synthesizing the flexibility found in NoCs with the efficiency of application-specific direct connections. By dissecting network traffic, EIGEN aims to allocate high-priority, large-volume communications to a direct connect network, while less critical traffic can utilize a flexible NoC. This bifurcation allows for leveraging specific advantages from both networks, providing a tailored solution to the identified challenges. EIGEN is architecturally composed of:

Various chiplets: Various functional chiplets are arranged, with heterogeneity visually indicated by varied colors.

Dual-layer Interconnect Network: Beneath the chiplets, the active interposer features two distinct interconnect layers; the first is application-aware switch-programable interconnection layer (**AspLayer**), ensuring high-throughput communication tailored to a specific application. The second layer, the dynamic packet-routing interconnection layer (**DynLayer**), utilizes a packet-switched network for auxiliary traffic. For visual clarity, as shown in Fig. 4, these layers are depicted as separate entities in our architectural diagrams, each tailored for specific types of traffic. The AspLayer is designed to handle major data flows, optimizing for low latency and high throughput, while the DynLayer enhances system flexibility and resilience by facilitating supplementary communications and memory access.

Both AspLayer and DynLayer connect to chiplets via μbumps or hybrid bonding, with different segments of the same connection array allocated respectively, completing the connection with interposer link through turn-out blocks [36]. Fig. 4(a) depicts the two interconnect architectures as separate upper and lower layers for visual clarity, although both interconnects share the same set of metal layers in the interposer. Dashed rectangles and colored lines are used to illustrate the system's differentiated connectivity for handling diverse traffic. Fig. 5 illustrates the EIGEN system architecture and its Silicon prototype. Fig. 5(a) and Fig. 5(b) depict the implemented prototype and its 3D cross-section, confirming EIGEN's physical feasibility. Fig. 5(c) details the architecture, where AspLayer supports low-latency, single-hop communication for high-performance applications, while DynLayer manages control and auxiliary data using flexible NoC transmission, enhancing scalability. The DynLayer also connects to the interposer SRAM, improving system performance with less external memory access while linking to memory controllers for external memory access. Additionally, the interposer integrates a Reinforcement Learning (RL) agent for flexible routing and a CPU for system initialization and configuration.

B. Application-aware Switch-programmable Layer

As depicted in Fig. 4(a), the **AspLayer** enables the configuration of a logical topology on the physical interconnect fabric, remaining transparent to the application. The physical structure of this interconnect comprises a 2D mesh topology consisting of topology switches and links, shown as rectangular blocks in Fig. 4(a). Similar to FPGA switches, these switches can be implemented using technologies such as multiplexers and transmission gates.

In Fig. 4(a), topology switches connect to four directional links: north, south, east, and west, enabling diverse port configurations. While a switch could theoretically connect any pair of links in all combinations, such an approach incurs $O(n^2)$ complexity, increasing area and power significantly. Hence, a hierarchical multiplexer approach is employed, allowing connections with a linear $O(n)$ cost increase. AspLayer supports multiple links, allowing bandwidth adjustments based on application needs. Unused switches and links are powered down via clock and power gating to reduce dynamic and static power consumption and enhance energy efficiency.

Through the programmable topology switches, different directional links can be selectively connected to create a globally optimized logical topology, which is optimized for different applications on the same physical fabric. Fig. 6 shows various AspLayer topology configurations for different applications. Fig. 6(a) shows a cmesh topology with minimal links and potential for further customization to improve efficiency. Fig. 6(b) illustrates a tree topology for homogeneous GPU integration, leveraging the high bandwidth of tree topology. Fig. 6(c) optimizes for AI applications with branch-structured data flows, facilitating efficient pipeline scheduling. Fig. 6(d) demonstrates diverse chiplet integration with strategic placement and custom connections, minimizing transmission paths, reducing latency, and enhancing energy

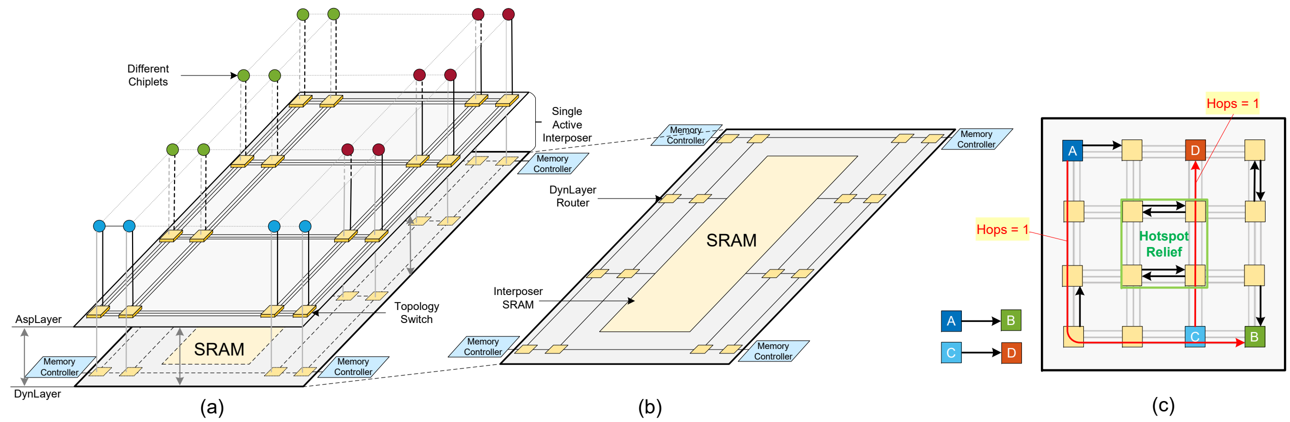

Fig. 4: The EIGEN interconnection architecture, drawn as a two-layer interconnection for clarity, with AspLayer (upper) and DynLayer (lower). (a) EIGEN overall interconnection architecture. (b) DynLayer interconnection, with memory integrated on the interposer. (c) An example of AspLayer interconnection is communication between connected nodes only requires one hop.

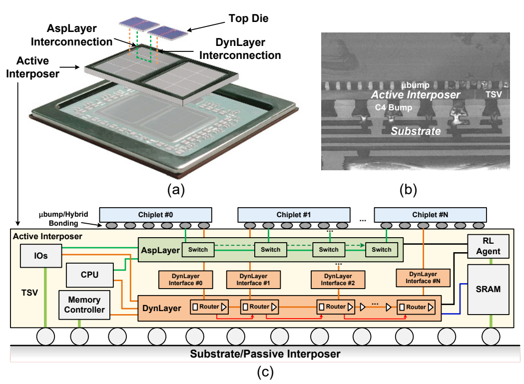

Fig. 5: Silicon prototype and detailed architecture of EIGEN. (a) The silicon prototype implementation of EIGEN [35]. (b) A 3D cross-section in EIGEN. (c) The comprehensive architecture of EIGEN.

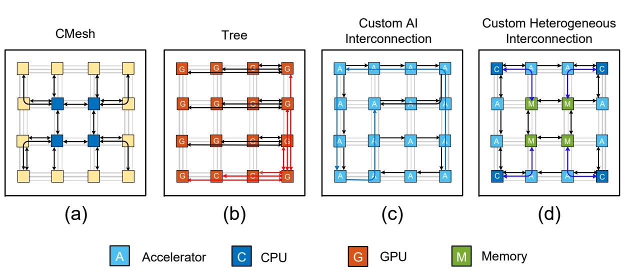

Fig. 6: AspLayer topology configuration cases. (a) to (d) represent different application scenarios.

efficiency. These configurations underscore the advancement over previous works [20]–[22]. Asplayer's programmable goal is not merely about reconfiguring topologies and connections but also supporting the reconfigurability of system functions and the heterogeneous integration of chiplets.

The programmable topology switch supports interconnects of various scales and topologies, facilitating heterogeneous chiplet integration and active interposer reuse, which is the key to economical chiplet-based systems. AspLayer can establish direct links between any two nodes (chiplets) with minimal network utilization impact. As shown in Fig. 4(c), a direct logical connection between chiplet a (purple) and chiplet b (green) requires only one hop, compared to six hops in a traditional NoC shown in Fig. 2(b) and (c). Similarly, chiplet c (blue) and d (red) are connected in the same way. The AspLayer offers significant benefits: it enables efficient, low-latency data transmission between long-distance chiplet pairs and reduces the overall number of hops in the system, optimizing link congestion and power consumption, and minimizing hotspots. By optimizing the topology based on application needs, the system's overall energy efficiency is improved. Additionally, Quality of Service (QoS) management can be achieved by simple hardware configuration without complex software intervention, reducing management costs.

C. Dynamic Packet-routing Interconnection Layer

For clarity, Fig. 4(a) positions the dynamic packet-routing network **DynLayer** at the lower tier of EIGEN's architecture. The DynLayer integrates SRAM directly onto the active interposer, taking advantage of the slower evolution of SRAM technology compared to logic processes. Since the interposer's process node is typically older than the chiplets, incorporating SRAM into the active interposer is both practical and beneficial, leveraging mature node technologies for cost-effectiveness. As depicted in Fig. 4(b), this configuration allows chiplets to access memory through short vertical links, reducing latency compared to traditional horizontal interconnects. Placing the memory centrally in the interposer helps balance latency disparities among chiplets. Furthermore, DynLayer's management of limited traffic communication reduces network congestion risks, mitigating memory access latency variability in NUMA systems caused by multi-hop and congestion. The system efficiently handles various traffic, including control signals and network status monitoring.

DynLayer routers are modified with an upward port and a control module for managing vertical and horizontal packet routing. This configuration uses source routing for DynLayer paths, where data from chiplets is directed to the DynLayer router with the destination address, completing the transmission within DynLayer. This dual-layer architecture integration improves the flexibility and efficiency of chiplet-to-chiplet and chiplet-to-memory communications.

D. *EIGEN Configuration and Management*

In this section, we describe the management mechanisms of the EIGEN, including system configuration and operational execution, routing framework, and deadlock avoidance.

1) EIGEN Configuration and Management: Based on the application running on EIGEN, we map the AspLayer topology, generate a bitfile for offline configuration, and program it into the AspLayer's configuration registers to set the topology for runtime. Notably, the AspLayer's interconnect topology remains unchanged for each execution instance. The CPU within the interposer manages system boot-up and module setup, subsequently activating the system control register to enable inter-chiplet communication. Control and status flits, typically compact, are transmitted through the DynLayer using unicast or multicast to synchronize and control system components or chiplets. With the AspLayer managing major traffic flows, the DynLayer handles these transmissions with minimal delay, ensuring prompt and effective control communication.

Route framework in EIGEN is managed by the RL agent, interacting with both the AspLayer and DynLayer, which are divided into subnetworks. Status from AspLayer is transmitted to the DynLayer router and combined with DynLayer data. This state information is then efficiently sent to the RL agent on a subnetwork basis. The RL agent determines routing decisions based on status, sending control commands back through DynLayer to the respective chiplets for route execution.

2) Deadlock Freedom Analysis: Circular channel dependencies and protocol dependencies can cause network deadlock. For protocol dependencies, request and reply packets are separated into AspLayer and DynLayer or different virtual networks within DynLayer. While each chiplet is independently verified deadlock-free, interconnecting multiple chiplets can introduce potential cyclic dependencies [37]. In EIGEN, three types of potential loops may form: (1) between chiplets and AspLayer, (2) between chiplets and DynLayer, and (3) between chiplets, AspLayer, and DynLayer. Using dimension-ordered routing ensures loops are not formed in DynLayer. We employ bubble flow control [38] on DynLayer to break loops (2) and (3). For loop (1), redundant channels are implemented on AspLayer to prevent deadlocks, inspired by the escape channel concept. This approach ensures deadlock-free routing in EIGEN.

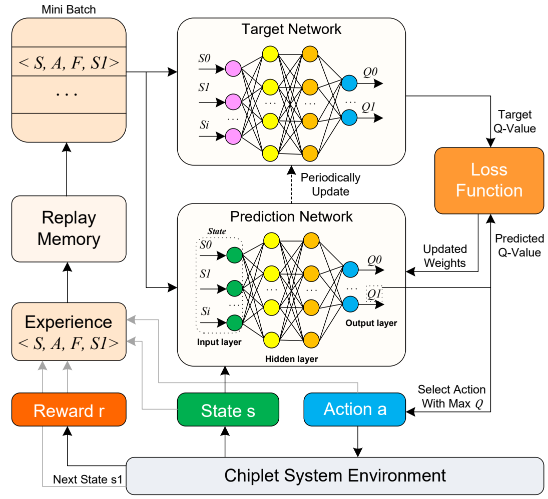

Fig. 7: The RL-based routing framework.

IV. Reinforcement Learning-Based Routing Framework

A. *Framework Introduction and Fundamentals*

The rising complexity in chiplet-based systems demands sophisticated routing strategies due to inefficiencies such as increased latency and energy usage with traditional methods. We propose an RL routing framework, tailored to optimize paths within our dual-interconnection architecture, aiming to improve application efficiency in heterogeneous environments. As shown in Fig. 5, it consists of two pathways: AspLayer for direct inter-chiplet communication and DynLayer for flexible packet routing. The AspLayer topology is conceptualized as graph $G(S, E)$, where $S = S_1, S_2, \ldots, S_N$ represents the set of switches, and $E = S \times S$ embodies the links between switches, $e_{(i,j)}$ is the link connecting the switches S_i and S_j such that $e_{(i,j)} \in E$. A route path $\mathcal{R}$ is a sequence of switches $S_{src} \to S_{j_1} \to \cdots \to S_{j_n} \to S_{dst}$ such that data is traversed from source switch S_{src} to destination S_{dst}. The source switch S_{src} generates data P at time t_1, which reaches at S_{dst} in time $t_{dst_{AspLayer}}$, which is defined as:

$$t_{dst_{AspLayer}} = \sum_{\forall S_i \in \mathcal{R}} t_{S_i} + t_l \tag{1}$$

DynLayer path is stated as $P(C, M, E)$, where M represents interposer memory. The specific DynLayer route path $\mathcal{R}$ is a sequence of chiplets and interposer memory $C_{src} \to (M_{interposer}) \to C_{internal} \to C_{dst}$. For the DynLayer path, the corresponding path delay $t_{dst_{DynLayer}}$ is:

$$t_{dst_{DynLayer}} = \begin{cases} t_{write} + t_{C2M} + t_{read} + t_{M2C} \\ n_{hops} \times t_{unit} \end{cases} \tag{2}$$

Where t_{write}, t_{read} represent the interposer memory read and write time. t_{C2M}, t_{M2C}, and t_{unit} represent the link delay between chiplets and memory, and the unit hop delay.

System performance is greatly affected by the average delay of data communication, where the average delay $T_{average}$ is defined as:

$$T_{average} = \frac{\sum_{i=1}^{m} t_{dst_{AspLayer\,i}} + \sum_{j=1}^{n} t_{dst_{DynLayer\,j}}}{m+n} \quad (3)$$

In the above scenario, we aim to design a routing algorithm to improve the chiplet-based system performance. The goal can be formally stated as: Based on the given $G(S, E)$, $P(C, M, E)$, $M_{interposer}$, C_{src}, C_{dst}, the goal is to use a congestion-controlled and load-balanced adaptive routing algorithm to find the optimal path for data communication from C_{src} to C_{dst} which minimizes the average delay $T_{average}$ and system communication overhead.

Fig. 7 shows the RL agent updates routing strategies in EIGEN through interactions based on status updates and rewards. The environment models interactions among chiplets, interposer, and memory, incorporating elements like switches, routers, and links to facilitate data flow. This enables the RL agent to assess the system's state accurately and adapt its strategies. Simulating real-world traffic trains the agent to handle various scenarios, enhancing the routing framework's adaptability and robustness.

B. Reinforcement Learning Strategy and Routing Decisions

The RL framework operates within a defined state and action space, which are crucial for making informed routing decisions. The algorithm uses the ϵ-greedy strategy and experience replay to select routing paths. The routing framework is defined as $< S, A, R, \phi >$, where S, A, R and ϕ represent state space, action space, reward, and the transition function.

1) State Space: The state space of our RL framework comprises detailed metrics from Table I, such as network traffic, memory demands, and AspLayer's topology, represented by a six-dimensional feature vector derived from packets, routers, and the interconnect architecture. Each unique routing decision correlates with distinct state features. At a given moment t, if a packet P is ready for routing at chiplet C, the state is defined as $S = [f_0, f_2, f_3, \ldots, f_{12}]$. This vector helps the RL agent choose the optimal routing path based on current conditions.

TABLE I: RL Framework Status Parameters

State Indicators	State attributes
Topology Metrics	Path chiplet IDs
	Current AspLayer topology
	Size of topology
Network Metrics	Number of data packets
	Router buffer utilization
	Switch throughput
	AspLayer interface buffer utilization
	DynLayer interface buffer utilization
	Current path utilization
Memory Metrics	Interposer memory port buffer utilization
	Utilization of interposer memory
	Chiplet memory miss
	Interpoer memory miss

- **Topology Metrics (f_{0-2}):** Topology information includes network size, AspLayer topology, and chiplet ID paths (id_{start} and id_{dst}), crucial for guiding routing decisions.
- **Network Metrics (f_{3-8}):** These metrics include intra-chiplet communication types (data and control), indicating different application phases. Interface buffer utilization is a key indicator of traffic load and congestion, with individual monitoring to accurately measure them.
- **Memory Metrics (f_{9-12}):** These metrics include the number of accesses to chiplet and interposer memory, memory utilization, and port usage, indicating computational demand and the availability of the DynLayer path.

2) Action Space: In the heterogeneous chiplet-based system, the action space for EIGEN is defined by a two-dimensional vector, $A = [a_1, a_2]$, representing two routing paths: **AspLayer Path** (a_1) for direct inter-chiplet communication and **DynLayer Path** (a_2) for flexible packet-routing, used for buffering or when AspLayer are busy. The action a_t chosen at time t determines the routing path for packet P, with Q values indicating the effectiveness of the selected action.

3) Reward: The reward mechanism in the RL routing framework is crucial for guiding the agent's actions. It aims to reduce packet latency and link utilization by providing positive feedback when the agent's decisions improve system performance. This propels the agent to prioritize pathways that amplify the efficacy of data throughput within the EIGEN.

$$reward = -\ T_{average}\ \times\ U_{link} \quad (4)$$

The reward is negatively proportional to the product of average packet latency and link utilization, encouraging agent to discover and exploit routing strategies to optimize indicators.

In the RL framework shown in Fig. 7, the RL agent interacts with the EIGEN system at discrete intervals, monitoring communications, gathering state features, and selecting optimal actions based on Q-values. Actions cause state transitions to s_{t+1}, and the resulting feedback refines the agent's strategy, optimizing cumulative rewards through continuous learning.

C. Training and Optimization Algorithm

The routing framework, detailed in Algorithm 1, begins by initializing the prediction and target networks to set initial weights and states. State information from the environment, captured by the `EnvObserve` function, includes static and dynamic data, feeding into RL agent. The agent uses an ϵ-greedy strategy to choose actions between times 1 to T, opting for either random action or the one with the highest Q-value, promoting exploration and avoiding local optima. Actions influence routing decisions, with corresponding rewards and state updates processed through `getreward` and `getnextstate` functions.

Due to the impracticality of maintaining a large Q-value table, we use a deep neural network to approximate Q-values, which features three hidden layers, uses Relu and linear activations for hidden and output layers, respectively, trained with mean square error (MSE) in `LossComp` and Adam optimization in `WeightUpdate`. Trained weights are loaded into the RL agent for inference. As shown in Fig. 7, training uses dual networks: a prediction network guiding actions and a target network calculating true Q-values for updating strategies based on the Bellman equation [39].

Algorithm 1: Proposed Flex RL algorithm

input: The state information $S = [f_0, f_1, ..., f_{12}]$
output: State-action value function $Q_w(s)$

Q, $\hat{Q} \leftarrow$ `IniNetwork`(w, $\hat{w}$);
`IniReplyMem`(M);
`IniState`($s_1 = [f_0, f_1, ..., f_{12}]$);
for $t \leftarrow 1$ **to** T **do**
  $s_t \leftarrow$ `EnvObserve`(*env*)
  Select an action a_t *using* ε*-greedy approach*;
  if *Probability* $< \varepsilon$ **then**
    $a_t =$ `RandomAction`(*seed*);
  end
  else $a_t = argmax_a Q_{wt}(s_t)$;
  `ExecuteAction`(a_t);
  $f_t \leftarrow$ `GetReward`(*env*) ;
  $s_{t+1} \leftarrow$ `GetNextState`(*env*);
  $M \leftarrow$ `StoreTuple`(s_t, a_t, s_{t+1}, f_t) ;
  for T ***in*** T_{train} **do**
    if `GetNumber`(M) $> M_{threshold}$ **then**
      `RandomSample`($transition(s_t, a_t, s_{t+1}, f_t)$);
      form mini-batch M_s;
      for $transition(s_t, a_t, s_{t+1}, f_t)$ *in* M_s **do**
        `TargetComp`($f_t, \hat{Q}, s_{i+1}, a_{i+1}, \hat{w}$);
        $L(w) \leftarrow$ `LossComp`(Q_w, S_t, a_t, y_i) ;
        `WeightUpdate`(Q);
      end
    end
  end
  for T ***in*** *the* T_{target} **do**
    $\hat{Q} \leftarrow$ `WeightCopy`(Q) ;
  end
end
return *the Q-value function from* Q_w;

The framework enhances stability through an experience replay memory that stores and samples experiences to avoid biases associated with sequential data. During training, both networks process batch data, with the prediction network estimating immediate Q-values and the target network focusing on future state Q-values. Discrepancies between these values prompt updates via gradient descent, while the target network periodically adjusts weights using `WeigthCopy` to stabilize learning. Feedback mechanisms allow the RL model to adapt to network conditions, making adjustments to reduce congestion or latency and ensuring the routing framework remains robust and efficient. Continuous optimization based on performance seeks to improve packet delivery, minimize congestion, and optimize resource use, aligning with the strategic objectives of the deployed RL model.

V. Experimental Validation

A. Experiment Setup and Methodology

To rigorously evaluate EIGEN's performance in 3D chiplet-based systems, we conduct simulations using three distinct types of traffic: synthetic, heterogeneous, and high-load traffic. Heterogeneous traffic simulations utilized an APU platform integrating gem5 [40] and a GPU model [41], adapting GARNET [42] to support EIGEN's architecture and allowing for custom network configurations. DynLayer naturally fits within GARNET, facilitating CPU and GPU task simulations on this platform. Synthetic traffic was also processed using gem5, while high-load traffic scenarios involved placing accelerators at each chiplet node, using a modified Booksim [43] and accelerator simulator [44] to emulate AI tasks. Adjustments were made to Booksim topology and Traffic Manager model to support the chiplet-active interposer architecture. The RL model is implemented on the aforementioned simulation platform and performs closed-loop simulations with specific hyperparameters: learning rate α at 0.05, discount factor γ at 0.9, and exploration rate ϵ at 0.04, detailed in Section V-D.

TABLE II: Benchmark Applications

Benchmarks	Applications
Synthetic	uniform, neighbor, shuffle, bit-reverse, hotspot, bit-complement, transpose, fan-out traffic
CPU	canneal, x264, sw, blackshones
GPU	hotspot, bfs, k-means, heartwall
AI	ResNet50 [45], MobileNet V3 [46], AlexNet [47], Yolo V2 [48], NCF_recommadation [49], Transformer [50], GoogleNet [51], AlphaZeroGo [52]

Benchmarks [45]–[52] used are shown in Table II, where the transformer model uses the T5-base model [50]. Evaluation employs various traffic patterns on an 8×8 network with 8-flit packet sizes under synthetic conditions. For CPU [53] and GPU traffic, the network setup included 64 CPU chiplets using the MESI protocol, each featuring an x86 out-of-order core, 32 KB private L1 cache, and 1 MB shared L2 cache. The GPU benchmark [54] consists of 12 CPUs and 52 GPUs, each GPU having 64 KB private L1 cache and scratch memory. AI benchmarks are conducted on 8×8 accelerator grid, with interposer memory set at 32 MB. For EIGEN's DynLayer, routers have a 3-virtual channel, 4-stage design, four flit buffers per channel, 128-bit link bandwidth, and hop/link latency of 1 and 2 cycles. EIGEN maintains equivalent bandwidth to other architectures, with a 2:1 ratio between AspLayer and DynLayer. We allocate one memory controller to each $4 \times N/4$ subnetwork within an $N \times N$ network (where $N = 4, 8, 12, \ldots, 32$) to meet bandwidth needs, prioritizing edge routers for initial placement but ensuring uniform distribution across large networks to optimize latency and balance loads. We use a built-in memory model [55] in gem5 with N channels and eight banks per channel. EIGEN's scalability was further evaluated through scale sensitivity analyses in 4×4 to 32×32 network size.

We thoroughly investigate interconnect architectures and examine the performance of EIGEN by comparing EIGEN with the following works:

1) **2.5D NoC [7]:** A 2.5D mesh inter-chiplet network derived from conventional 2D mesh with die-to-die interface bridge adjacent chiplets, as the baseline.

2) **Bypath [17]:** Bypath adds express links connecting distant routers in a conventional mesh topology, which helps

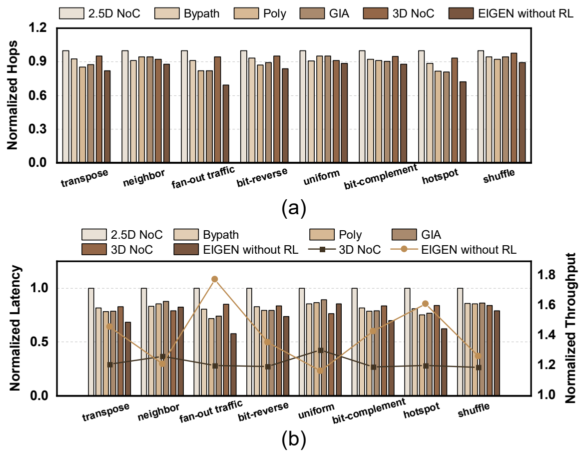

Fig. 8: Performance analysis based on synthetic traffic. (a) Normalized hop count. (b) Normalized latency and throughput.

bypass hotspots and alleviate system congestion.

3) **Poly [22]:** Poly enhances the original NoC with configurable external switches, enabling static topology customization tailored for specific applications.

4) **GIA [16]:** GIA introduces bypass channels within routers, offering the flexibility to switch between standard and bypass routing modes.

5) **3D NoC:** 3D NoC bridges top dies and base die through stacking interfaces with two-layer NoC and interposer memory to compare with EIGEN.

6) **EIGEN without RL:** EIGEN integrates an application-aware topology and a flexible NoC with integrated memory on the interposer to facilitate memory access.

7) **EIGEN with RL:** EIGEN advances application-specific routing optimization through the RL-based framework, enhancing route selection for improved network performance.

B. *Performance Analysis*

1) Packet Latency and Throughput Analysis: Fig. 8(b) shows EIGEN's performance across various synthetic traffic patterns, with 16.79% to 77.91% saturation throughput increase (right y-axis) and 14.38% to 42.32% average latency reduction (left y-axis). These gains are attributed to EIGEN's adaptability to traffic characteristics. EIGEN achieves latency reductions of 19.68% to 42.33% in fan-out traffic. Bypath, Poly, and GIA mitigate congestion by offering customized pathways, while 3D NoC's additional routes increase path diversity. Although EIGEN exhibits slightly higher latency than general 3D NoC when handling random and neighbor traffic patterns, the minor increase does not detract from its overall performance and effectiveness in specialized applications. EIGEN performs well with predictable traffic patterns. As the regularity increases, the performance improvements become more and more significant. Based on this observation, an optimal AspLayer topology can be determined if prior knowledge of network traffic, such as patterns in neural networks, is available.

Fig. 9(c) illustrates EIGEN's performance in CPU and GPU applications, reducing average latency (left y-axis) by 12.98% to 70.6%. Compared to 2.5D NoC, 3D NoC offers additional paths and low-latency memory access via vertical links from interposer memory. Poly uses configurable switches outside routers for flexibility but adds inherent delays, less efficient than GIA. With AspLayer's tailored customization and circuit-switched architecture, EIGEN achieves lower latency without complex bypass logic. In memory-intensive CPU benchmarks (sw, x264, ca), 3D NoC outperforms Poly and GIA due to its flexible NoC and efficient memory access, acting as a last-level cache and mitigating unbalanced memory access in NUMA systems. The polylines (right y-axis) in Fig. 9(c) indicate EIGEN's throughput optimization over 3D NoC, demonstrating the benefits of custom network topology.

Fig. 10(c) and Fig. 11 illustrate latency and the relationship between system latency and packet injection rate in AI benchmarks, which represents another focal evaluation point: high-load applications sensitive to throughput. EIGEN reduces latency (left y-axis) by 38.72% to 67.17% and boosts throughput (right y-axis) by 1.74x to 2.39x, utilizing application-aware topology customization and circuit-switched links for efficient data transmission. While Poly and GIA offer network customization, Poly's external switch design causes sequential data passage and bandwidth contention, while GIA's channel-specific assignments may lead to conflicts. Despite GIA's bypass channels, it cannot match AspLayer's fully customizable topologies. Furthermore, the advantages are less pronounced in applications like AlexNet, where network communication overhead is lower, underscoring EIGEN's benefits for traffic-intensive tasks.

2) Hops Analysis: Fig. 8(a), Fig. 9(a), and Fig. 10(a) show EIGEN's ability to reduce hop counts by 10.65% to 52.32% across various applications. Mesh-based systems like the baseline, Bypath, and 3D NoC typically result in higher hop counts due to larger network diameters. While Bypath provides shortcut paths, they are limited and lack customization flexibility. 3D NoC offers greater path diversity, reducing detours from congestion and thus hop reduction, particularly effective in uniform and neighbor traffic patterns. For memory-intensive applications (x264, sw, canneal), 3D NoC leverages interposer memory with short vertical links to minimize hops. Poly and GIA are effective in defined patterns but struggle with random traffic, resulting in increased hops. They perform slightly behind 3D NoC in high-memory scenarios but have advantages in AI benchmarks. EIGEN combines programmable topology and flexible packet-routing architecture, thus handling minimal hops, and further enhanced by its RL-based routing, achieving additional hop reductions of 2.76% to 6.87%.

3) Runtime Analysis: Fig. 9(b) and Fig. 10(b) show normalized execution time across various tasks. EIGEN outperforms other designs in heterogeneous and high-load benchmarks, reducing execution times by 4.79% to 9.32%. EIGEN achieves an average runtime reduction of 4.23% compared to 3D NoC, with more significant reductions in high-load applications like Transform and YOLO, and traffic-intensive applications like Hotspot. EIGEN's execution times are 6.96% shorter than Bypath due to its ability to provide ample long-distance channels

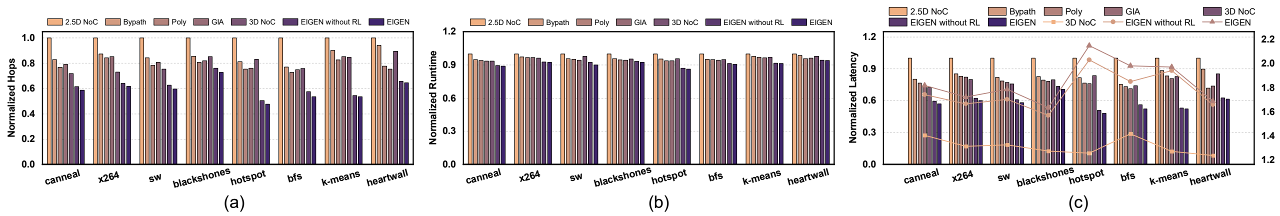

Fig. 9: Performance analysis based on CPU and GPU benchmark. (a) Normalized hops count. (b) Normalized runtime. (c) Normalized latency (left y-axis) and throughput (right y-axis).

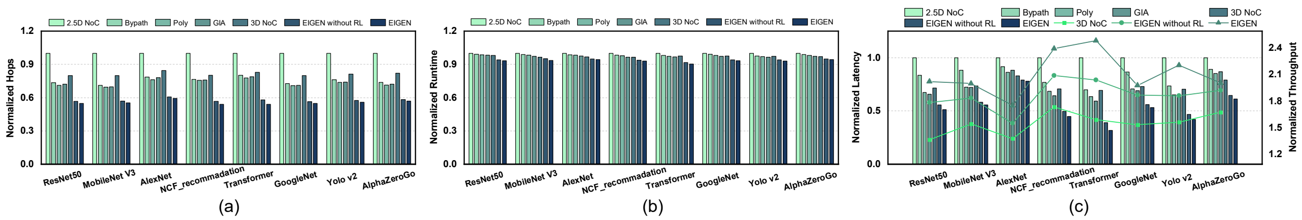

Fig. 10: Performance analysis based on AI benchmark. (a) Normalized hops count. (b) Normalized runtime. (c) Normalized latency (left y-axis) and throughput (right y-axis).

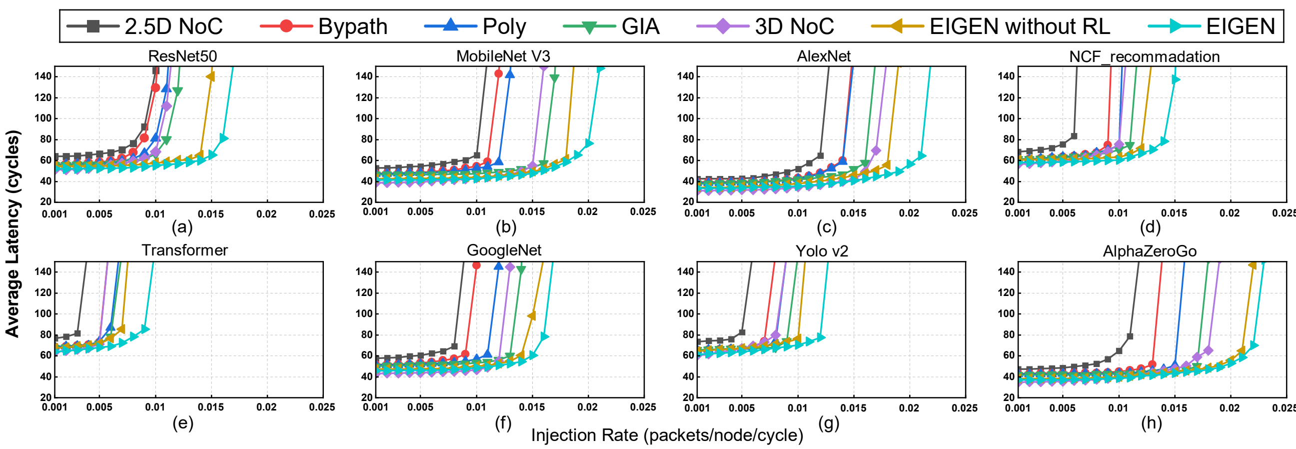

Fig. 11: Latency comparison based on AI benchmark as the load increases.

and customize network topologies. The separation of circuit-switching structures from routers in EIGEN's AspLayer, combined with its integration with the flexible DynLayer, leads to an average execution time reduction of 4.97% and 4.41% compared to Poly and GIA, respectively. Moreover, EIGEN's RL routing framework reduces latency by an additional 1.28% over EIGEN without RL, emphasizing its effectiveness in minimizing network latency.

4) Congestion and Hotspot Alleviation: To assess traffic distribution and identify communication bottlenecks, we present link utilization heatmaps in Fig. 12 for ResNet under near-saturation conditions. Darker reds indicate higher utilization, highlighting queuing latency and potential congestion. The 2.5D NoC (Fig. 12(a)) experiences the most severe issues, while 3D NoC (Fig. 12(e)) reduces link utilization by providing alternative and flexible routes, but significant hotspots remain. Bypath (Fig. 12(b)) introduces express channels to alleviate congestion but slightly increases link utilization around hotspots. Poly and GIA (Fig. 12(c) and Fig. 12(d)) offer further customization to mitigate hotspot issues. EIGEN

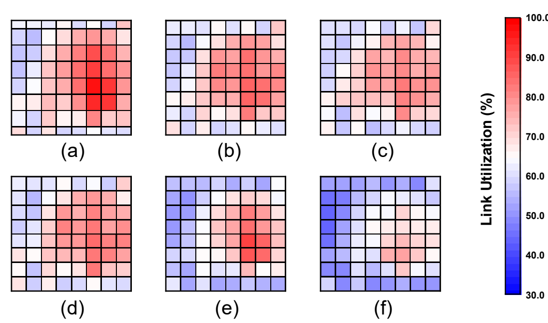

Fig. 12: Network link utilization heatmap. (a) to (f) represent baseline, Bypath, Poly, GIA, 3D NoC and EIGEN.

(Fig. 12(f)), with comprehensive topology customization and low-latency interposer memory access, achieves more even traffic distribution.

C. Sensitivity Analysis

Section V-B evaluated latency, throughput, and hop counts for 8×8 network setup. To evaluate EIGEN's scalability, we

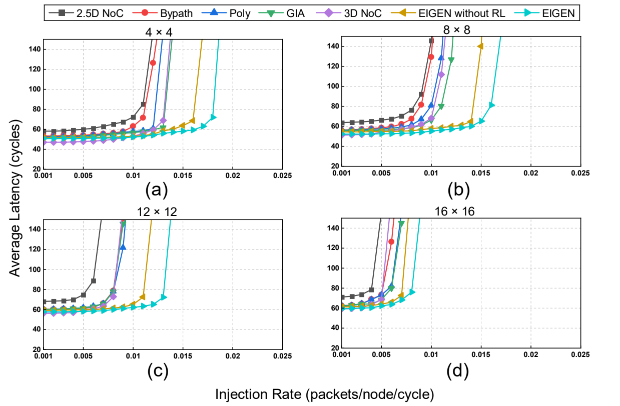

Fig. 13: Sensitivity analysis on network size of latency-throughput relationship for ResNet 50.

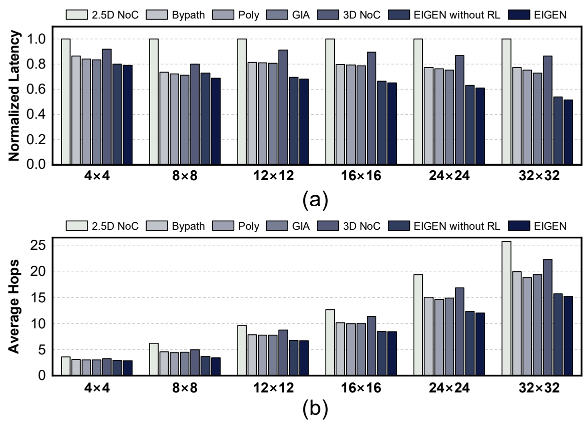

Fig. 14: Sensitivity study on network size. (a) Normalized latency. (b) Average Hops.

tested networks from 4×4 to 32×32 with the same ResNet50 benchmark. Fig. 13 shows the relationship between latency and injection rate across different network sizes. As the network grows, EIGEN consistently maintains and even enhances its advantage, achieving the highest throughput. This sustained performance stems from EIGEN's fully adaptable architecture, which minimizes network diameter and simplifies connections between distant nodes, which grows increasingly vital as the chiplet number increases, due to larger networks facing more severe challenges with unbalanced traffic distribution and heightened congestion risks. Additionally, EIGEN's AspLayer lowers latency and reduces hops as networks expand. Fig. 14 details latency and hop counts, showing latency reductions ranging from 20.97% to 47.42% compared to 2.5D NoC, and from 5.19% to 29.28% compared to GIA. Fig. 14(b) shows that while all architectures see an increase in hop counts, EIGEN's increment is the smallest, widening the performance gap. In a 32×32 chiplets configuration, EIGEN achieves an average hop count of 15.68, less than 25.76 in 2.5D NoC, demonstrating EIGEN's good scalability.

D. RL Analysis

EIGEN's dual-layer interconnect architecture, comprising AspLayer and DynLayer, offers two distinct routing paths crucial for optimizing system performance regarding latency, throughput, and power consumption. The RL framework can select the optimal path based on network states and reward feedback. The framework uses three hyperparameters: learning rate α, discount factor γ, and exploration rate ϵ. Initial parameter loop exploration revealed the importance of the ϵ-greedy algorithm in balancing exploration and exploitation, with an optimal ϵ setting of 0.04. A discount factor of 0.9 and a learning rate of 0.05 were found to provide the best balance for convergence and performance stability.

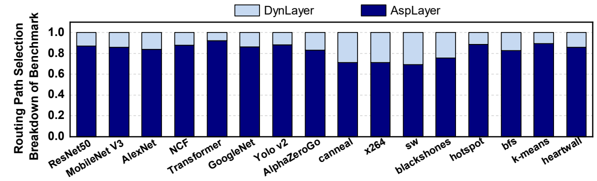

Fig. 15: Path selection breakdown of different benchmarks based on RL framework.

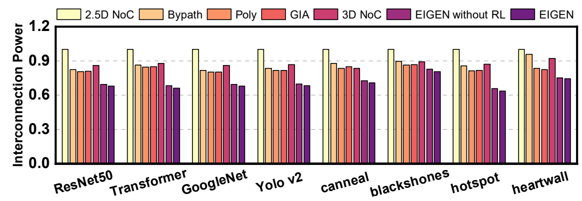

Fig. 16: Normalized energy analysis of different benchmarks.

In this framework, traffic routing across an 8×8 network, with Fig. 15 showing a preference for the AspLayer, which handles 79.85% of the traffic for its faster data transmission. CPU benchmarks, often involving sparse communications, favor DynLayer at 28.17%, with memory-intensive applications like x264 and sw showing even higher usage at 28.95% and 30.68% due to interposer memory integration. GPU and AI applications, requiring higher throughput, predominantly use AspLayer, with rates of 82.53% and 92.35%. The RL framework optimizes routing, reducing latency by 4.22%, 3.97%, and 7.78% for CPU, GPU, and AI tasks, respectively, compared to EIGEN without RL. Scalability analyses from 4×4 to 32×32 networks show average latency reductions of 1.37% to 4.22%, highlighting the framework's effectiveness in enhancing performance by leveraging both custom topology and flexible packet-routing capabilities.

The RL framework's exploration shows that AspLayer is preferred for large-scale, high-density data traffic, while DynLayer handles scattered, low-volume, and dynamic traffic. This traffic division leverages the efficiency of circuit-switched networks for large data and the flexibility of packet-switched networks for auxiliary traffic, offering practical and adaptable traffic management for chiplet-based systems.

E. Energy Consumption Analysis

Fig. 16 shows energy analysis using DSENT [56], showing that EIGEN reduces overall energy consumption by 2.37% to 29.98% compared to selected work, respectively. EIGEN's energy efficiency stems from both dynamic and static power savings. With relatively low average traffic, network power consumption is mainly static power. The shorter execution time of EIGEN reduces system operation duration, thus lowering static power. Regarding dynamic power, EIGEN's AspLayer customizes topologies to meet application demands, minimizing intermediate hops and avoiding energy costs related to data forwarding and buffering. Its circuit-switching architecture also eliminates the need for multi-stage router pipelines, such as channel allocation and routing computation, further reducing dynamic power consumption.

F. Overhead Analysis

1) Metal Wiring Constraints Analysis: EIGEN maintains the same total link bandwidth as other interconnect architectures for fair comparison, with bandwidth allocated between the AspLayer and DynLayer. As a result, EIGEN's links require no additional routing resources. We also assess whether routing demands stay within the maximum wire capacity limits [57], [58]. Although increased routing resources from more metal layers in the active interposer [59], [60], we evaluate EIGEN using the 28nm metal model with twelve layers (M1-M12) available [61]. Intermediate metal layers (M2-M6) feature 50 nm wires width, 50 nm wire spacing, and 100 nm pitch; higher metal layers (M7-M10) have 100 nm wires and 200 nm pitch. For a 0.25 mm^2 tile, a 256-bit bidirectional link requires only 2.05% of intermediate or 5.12% of high-layer resources, within routing constraints. For routing optimization, the AspLayer is customized for specific applications and manages most traffic with higher bandwidth allocation. We assign high metal layers to AspLayer to reduce signal attenuation and improve signal integrity, while DynLayer mainly uses lower metal layers.

2) Area Overhead Analysis: We synthesized EIGEN using the Design Compiler with the 28 nm library. The DynLayer routers occupy 104338 μm^2. EIGEN introduces an additional circuit-switched network, the AspLayer. However, the switches within the AspLayer do not require large buffers, arbitration, and routing logic, resulting in reduced hardware complexity and area overhead. According to synthesis reports, switch logic consumes only 14152 μm^2, 13.56% of the DynLayer router area, while AspLayer offers double the bandwidth of DynLayer. As a result, EIGEN would require less area than the baseline.

3) Runtime Overhead Analysis: In EIGEN, AspLayer's topology configuration and system initialization are completed before runtime, eliminating runtime overhead and enhancing efficiency. The additional time overhead mainly comes from routing framework latency, involving request/action transmission and routing computation. State information is transmitted at the subnetwork level, and actions, are both small in scale and contribute minimal delay. The RL agent's trained RL model maintains a consistent decision latency. Path selection latency accounts for 1.98% to 3.26% of total system latency for 8×8 to 32×32 configurations, respectively. As shown in Fig. 9 and Fig. 10, although additional overheads, EIGEN overcomes the dual-layer interconnect management overhead and provides overall communication benefits.

4) Discussion on Thermal Management: In 3D IC technology, managing thermal dissipation is crucial as higher power densities from increased component integration in limited volumes. This often leads to hotspots and potential thermal runaway [62], as each chip layer needs to dissipate heat through its top surface. In interposer-chiplet systems, a single layer of chiplets dissipates heat through the packaging top surface, similar to flip-chip packaging circuits. While active interposers incorporate active components that modestly increase the power budget, chiplets consume the most power. Power density primarily drives thermal complexity. Active interposer in EIGEN maintains a low power density of approximately 60 mW/mm^2, sourced mainly from SRAM and interconnects, rarely causing hotspots. Conversely, top dies can have 2-5 times higher power densities and are prone to hotspots from compute-intensive modules [11]. Prior research offers several effective solutions to these thermal challenges [62]–[76]. Additionally, EIGEN improves data communication efficiency, ensures more uniform traffic distribution, and mitigates data hotspots, thereby helping to alleviate thermal challenges.

We implemented a silicon prototype based on EIGEN architecture to validate its thermal management. Thermal analysis of the interposer, top dies, and cooling components are performed to maintain temperature under 100 °C. For system dissipating a few hundred watts, forced air and TIM (Thermal Interface Material) heatsinks are used in our prototype. Notably, for higher power dissipation, enlarging the heatsink or using liquid cooling can further optimize temperature control.

VI. Conclusion

Chiplet-based 3DICs enhance modularity and yield but face challenges in handling heterogeneous, high-load traffic, necessitating flexible topologies, low-latency communication, and efficient congestion management in NOAI. To address these issues, we propose EIGEN, a dual-layer interconnect architecture for chiplet-interposer systems, featuring AspLayer for efficient chiplet communication and DynLayer for flexible memory access and auxiliary traffic. Integrated with the RL routing framework routing, EIGEN optimizes traffic flow and enhances system performance. Our evaluation shows EIGEN achieves 67.16% latency reduction, 53.89% hops reduction, and 11.21% runtime reduction compared with the SOTA work. The RL framework's path exploration shows that large, dense data traffic prefers the AspLayer, while smaller, dynamic traffic favors the DynLayer. This traffic division leverages the strengths of both circuit-switched and packet-switched networks for efficient management in chiplet-based systems.

Acknowledgment

We thank anonymous reviewers for their insightful comments. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants 62322404 and 62304047, in part by the National Key Research and Development Program of China under Grant 2023YFB4404402.

REFERENCES

[1] A. Kannan, N. E. Jerger, and G. H. Loh, "Enabling interposer-based disintegration of multi-core processors," in *Proceedings of the 48th international symposium on Microarchitecture*, 2015, pp. 546–558.

[2] M.-S. Lin, T.-C. Huang, C.-C. Tsai, K.-H. Tam, K. C.-H. Hsieh, C.-F. Chen, W.-H. Huang, C.-W. Hu, Y.-C. Chen, S. K. Goel *et al.*, "A 7-nm 4-ghz arm[1]-core-based cowos[1] chiplet design for high-performance computing," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 4, pp. 956–966, 2020.

[3] P. Fotouhi, S. Werner, J. Lowe-Power, and S. B. Yoo, "Enabling scalable chiplet-based uniform memory architectures with silicon photonics," in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 222–334.

[4] G. Yeric, "Moore's law at 50: Are we planning for retirement?" in *2015 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2015, pp. 1–1.

[5] A. Coskun, F. Eris, A. Joshi, A. B. Kahng, Y. Ma, and V. Srinivas, "A cross-layer methodology for design and optimization of networks in 2.5 d systems," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2018, pp. 1–8.

[6] S. Naffziger, N. Beck, T. Burd, K. Lepak, G. H. Loh, M. Subramony, and S. White, "Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 57–70.

[7] D. Stow, I. Akgun, and Y. Xie, "Investigation of cost-optimal network-on-chip for passive and active interposer systems," in *2019 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP)*. IEEE, 2019, pp. 1–8.

[8] W. Gomes, A. Koker, P. Stover, D. Ingerly, S. Siers, S. Venkataraman, C. Pelto, T. Shah, A. Rao, F. O'Mahony, E. Karl, L. Cheney, I. Rajwani, H. Jain, R. Cortez, A. Chandrasekhar, B. Kanthi, and R. Koduri, "Ponte vecchio: A multi-tile 3d stacked processor for exascale computing," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 42–44.

[9] A. Smith, E. Chapman, C. Patel, R. Swaminathan, J. Wuu, T. Huang, W. Jung, A. Kaganov, H. McIntyre, and R. Mangaser, "Amd instincttm mi300 series modular chiplet package – hpc and ai accelerator for exa-class systems," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, 2024, pp. 490–492.

[10] D. Stow, Y. Xie, T. Siddiqua, and G. H. Loh, "Cost-effective design of scalable high-performance systems using active and passive interposers," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2017, pp. 728–735.

[11] P. Vivet, E. Guthmuller, Y. Thonnart, G. Pillonnet, C. Fuguet, I. Miro-Panades, G. Moritz, J. Durupt, C. Bernard, D. Varreau *et al.*, "Intact: A 96-core processor with six chiplets 3d-stacked on an active interposer with distributed interconnects and integrated power management," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 1, pp. 79–97, 2020.

[12] P. Coudrain, J. Charbonnier, A. Garnier, P. Vivet, R. Vélard, A. Vinci, F. Ponthenier, A. Farcy, R. Segaud, P. Chausse *et al.*, "Active interposer technology for chiplet-based advanced 3d system architectures," in *2019 IEEE 69th Electronic Components and Technology Conference (ECTC)*. IEEE, 2019, pp. 569–578.

[13] G. H. Loh, N. E. Jerger, A. Kannan, and Y. Eckert, "Interconnect-memory challenges for multi-chip, silicon interposer systems," in *Proceedings of the 2015 international symposium on Memory Systems*, 2015, pp. 3–10.

[14] N. E. Jerger, A. Kannan, Z. Li, and G. H. Loh, "Noc architectures for silicon interposer systems: Why pay for more wires when you can get them (from your interposer) for free?" in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014, pp. 458–470.

[15] S. Bharadwaj, J. Yin, B. Beckmann, and T. Krishna, "Kite: A family of heterogeneous interposer topologies enabled via accurate interconnect modeling," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.

[16] F. Li, Y. Wang, Y. Cheng, Y. Wang, Y. Han, H. Li, and X. Li, "Gia: A reusable general interposer architecture for agile chiplet integration," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.

[17] U. Y. Ogras and R. Marculescu, "Application-specific network-on-chip architecture customization via long-range link insertion," in *ICCAD-2005. IEEE/ACM International Conference on Computer-Aided Design, 2005.* IEEE, 2005, pp. 246–253.

[18] C.-H. O. Chen, S. Park, T. Krishna, S. Subramanian, A. P. Chandrakasan, and L.-S. Peh, "Smart: A single-cycle reconfigurable noc for soc applications," in *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2013, pp. 338–343.

[19] H. Zheng, K. Wang, and A. Louri, "Adapt-noc: A flexible network-on-chip design for heterogeneous manycore architectures," in *2021 IEEE international symposium on high-performance computer architecture (HPCA)*. IEEE, 2021, pp. 723–735.

[20] M. B. Stensgaard and J. Sparsø, "Renoc: A network-on-chip architecture with reconfigurable topology," in *Second ACM/IEEE International Symposium on Networks-on-Chip (nocs 2008)*. Ieee, 2008, pp. 55–64.

[21] M. Modarressi, A. Tavakkol, and H. Sarbazi-Azad, "Application-aware topology reconfiguration for on-chip networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 19, no. 11, 2010.

[22] M. M. Kim, J. D. Davis, M. Oskin, and T. Austin, "Polymorphic on-chip networks," *ACM SIGARCH Computer Architecture News*, vol. 36, no. 3, pp. 101–112, 2008.

[23] B. Munger, K. Wilcox, J. Sniderman, C. Tung, B. Johnson, R. Schreiber, C. Henrion, K. Gillespie, T. Burd, H. Fair *et al.*, "“zen 4": The amd 5nm 5.7 ghz x86-64 microprocessor core," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 38–39.

[24] H. Jiang, "Intel's ponte vecchio gpu: Architecture, systems & software," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 2022, pp. 1–29.

[25] I. K. Ganusov, M. A. Iyer, N. Cheng, and A. Meisler, "Agilex™ generation of intel® fpgas," in *2020 IEEE Hot Chips 32 Symposium (HCS)*. IEEE Computer Society, 2020, pp. 1–26.

[26] M. Hong and L. Xu, "Br100 gpgpu: Accelerating datacenter scale ai computing," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 2022, pp. 1–22.

[27] C.-C. Lee, C. Hung, C. Cheung, P.-F. Yang, C.-L. Kao, D.-L. Chen, M.-K. Shih, C.-L. C. Chien, Y.-H. Hsiao, L.-C. Chen *et al.*, "An overview of the development of a gpu with integrated hbm on silicon interposer," in *2016 IEEE 66th Electronic Components and Technology Conference (ECTC)*. IEEE, 2016, pp. 1439–1444.

[28] J. H. Lau, "Recent advances and trends in advanced packaging," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 12, no. 2, pp. 228–252, 2022.

[29] D. Velenis, M. Detalle, G. Hellings, M. Scholz, E. J. Marinissen, G. Van der Plas, A. La Manna, A. Miller, D. Linten, and E. Beyne, "Processing active devices on si interposer and impact on cost," in *2015 International 3D Systems Integration Conference (3DIC)*. IEEE, 2015, pp. TS11–2.

[30] W. Gomes, S. Morgan, B. Phelps, T. Wilson, and E. Hallnor, "Meteor lake and arrow lake intel next-gen 3d client architecture platform with foveros," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 2022, pp. 1–40.

[31] T. Jia, P. Mantovani, M. C. Dos Santos, D. Giri, J. Zuckerman, E. J. Loscalzo, M. Cochet, K. Swaminathan, G. Tombesi, J. J. Zhang *et al.*, "A 12nm agile-designed soc for swarm-based perception with heterogeneous ip blocks, a reconfigurable memory hierarchy, and an 800mhz multi-plane noc," in *ESSCIRC 2022-IEEE 48th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2022, pp. 269–272.

[32] T. Vijayaraghavan, Y. Eckert, G. H. Loh, M. J. Schulte, M. Ignatowski, B. M. Beckmann, W. C. Brantley, J. L. Greathouse, W. Huang, A. Karunanithi *et al.*, "Design and analysis of an apu for exascale computing," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 85–96.

[33] T. Wang, F. Feng, S. Xiang, Q. Li, and J. Xia, "Application defined on-chip networks for heterogeneous chiplets: An implementation perspective," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 1198–1210.

[34] N. Banerjee, P. Vellanki, and K. S. Chatha, "A power and performance model for network-on-chip architectures," in *Proceedings Design, Automation and Test in Europe Conference and Exhibition*, vol. 2. IEEE, 2004, pp. 1250–1255.

[35] B. Jiao, H. Zhu, Y. Zeng, Y. Li, J. Liao, S. Jia, Z. Chen, M. Tian, J. Zhu, D. Wen, Y. Wang, Y. Wang, J. Xu, F. Wang, J. Tao, C. Chen, Q. Liu, and M. Liu, "Shinsa: A 586mm² reusable active tsv interposer with programmable interconnect fabric and 512mb 3d underdeck memory," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2025, in press.

[36] B. Jiao, L. Xu, X. Yu, H. Yang, H. Zhu, Y. Wang, J. Zhu, D. Wen, L. Wang, J. Tao, C. Chen, Y. Han, Q. Liu, N. Sun, and M. Liu, "Fpia: Communication-aware multi-chiplet integration with field-programmable interconnect fabric on reusable silicon interposer," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[37] J. Yin, Z. Lin, O. Kayiran, M. Poremba, M. S. B. Altaf, N. E. Jerger, and G. H. Loh, "Modular routing design for chiplet-based systems," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 726–738.

[38] L. Chen and T. M. Pinkston, "Worm-bubble flow control," in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2013, pp. 366–377.

[39] M. A. Wiering and M. Van Otterlo, "Reinforcement learning," *Adaptation, learning, and optimization*, vol. 12, no. 3, p. 729, 2012.

[40] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.

[41] B. M. Beckmann and A. Gutierrez, "The amd gem5 apu simulator: Modeling heterogeneous systems in gem5," in *Tutorial at the International Symposium on Microarchitecture (MICRO)*, 2015.

[42] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "Garnet: A detailed on-chip network model inside a full-system simulator," in *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 2009, pp. 33–42.

[43] N. Jiang, D. U. Becker, G. Michelogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, "A detailed and flexible cycle-accurate network-on-chip simulator," in *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2013, pp. 86–96.

[44] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "Scale-sim: Systolic cnn accelerator simulator," *arXiv preprint arXiv:1811.02883*, 2018.

[45] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[46] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1314–1324.

[47] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[48] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 7263–7271.

[49] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th international conference on world wide web*, 2017, pp. 173–182.

[50] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[51] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.

[52] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton *et al.*, "Mastering the game of go without human knowledge," *nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[53] C. Bienia and K. Li, "Parsec 2.0: A new benchmark suite for chip-multiprocessors," in *Proceedings of the 5th Annual Workshop on Modeling, Benchmarking and Simulation*, vol. 2011, 2009, p. 37.

[54] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE international symposium on workload characterization (IISWC)*. Ieee, 2009, pp. 44–54.

[55] A. Hansson, N. Agarwal, A. Kolli, T. Wenisch, and A. N. Udipi, "Simulating dram controllers for future system architecture exploration," in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2014, pp. 201–210.

[56] C. Sun, C.-H. O. Chen, G. Kurian, L. Wei, J. Miller, A. Agarwal, L.-S. Peh, and V. Stojanovic, "Dsent-a tool connecting emerging photonics with electronics for opto-electronic networks-on-chip modeling," in *2012 IEEE/ACM Sixth International Symposium on Networks-on-Chip*. IEEE, 2012, pp. 201–210.

[57] J. Balfour and W. J. Dally, "Design tradeoffs for tiled cmp on-chip networks," in *ACM International conference on supercomputing 25th anniversary volume*, 2006, pp. 390–401.

[58] A. Ceyhan, M. Jung, S. Panth, S. K. Lim, and A. Naeemi, "Impact of size effects in local interconnects for future technology nodes: A study based on full-chip layouts," in *IEEE International Interconnect Technology Conference*. IEEE, 2014, pp. 345–348.

[59] B. Bohnenstiehl, A. Stillmaker, J. Pimentel, T. Andreas, B. Liu, A. Tran, E. Adeagbo, and B. Baas, "A 5.8 pj/op 115 billion ops/sec, to 1.78 trillion ops/sec 32nm 1000-processor array," in *2016 IEEE Symposium on VLSI Circuits (VLSI-Circuits)*. IEEE, 2016, pp. 1–2.

[60] W.-H. Liu, T.-K. Chien, and T.-C. Wang, "Metal layer planning for silicon interposers with consideration of routability and manufacturing cost," in *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2014, pp. 1–6.

[61] S. Yang, J. Sheu, M. Ieong, M. Chiang, T. Yamamoto, J. Liaw, S. Chang, Y. Lin, T. Hsu, J. Hwang *et al.*, "28nm metal-gate high-k cmos soc technology for high-performance mobile applications," in *2011 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 2011, pp. 1–5.

[62] C. Torregiani, B. Vandevelde, H. Oprins, E. Beyne, and I. De Wolf, "Thermal analysis of hot spots in advanced 3d-stacked structures," in *2009 15th International Workshop on Thermal Investigations of ICs and Systems*. IEEE, 2009, pp. 56–60.

[63] P. Birbarah, T. Gebrael, T. Foulkes, A. Stillwell, A. Moore, R. Pilawa-Podgurski, and N. Miljkovic, "Water immersion cooling of high power density electronics," *International Journal of Heat and Mass Transfer*, vol. 147, p. 118918, 2020.

[64] K. P. Drummond, D. Back, M. D. Sinanis, D. B. Janes, D. Peroulis, J. A. Weibel, and S. V. Garimella, "Characterization of hierarchical manifold microchannel heat sink arrays under simultaneous background and hotspot heating conditions," *International Journal of Heat and Mass Transfer*, vol. 126, pp. 1289–1301, 2018.

[65] G. Refai-Ahmed, H. Do, Y. Hadad, S. Rangarajan, B. G. Sammakia, V. Gektin, and T. Cader, "Establishing thermal air-cooled limit for high performance electronics devices," in *2020 IEEE 22nd Electronics Packaging Technology Conference (EPTC)*. IEEE, 2020, pp. 347–354.

[66] A. Z. Benelhaouare, A. Oukaira, M. Oumlaz, and A. Lakhssassi, "Efficient thermal management strategies for 3d-sip architectures," in *2024 18th International Conference on Ubiquitous Information Management and Communication (IMCOM)*. IEEE, 2024, pp. 1–5.

[67] C. Serafy, A. Srivastava, and D. Yeung, "Unlocking the true potential of 3d cpus with micro-fluidic cooling," in *Proceedings of the 2014 international symposium on Low power electronics and design*, 2014, pp. 323–326.

[68] G. Chen, J. Kuang, Z. Zeng, H. Zhang, E. F. Young, and B. Yu, "Minimizing thermal gradient and pumping power in 3d ic liquid cooling network design," in *Proceedings of the 54th annual design automation conference 2017*, 2017, pp. 1–6.

[69] Y. Zhang and M. Bakir, "Independent interlayer microfluidic cooling for heterogeneous 3d ic applications," *Electronics Letters*, vol. 49, no. 6, pp. 404–406, 2013.

[70] Y. Zhang, A. Dembla, and M. S. Bakir, "Silicon micropin-fin heat sink with integrated tsvs for 3-d ics: Tradeoff analysis and experimental testing," *IEEE Transactions on Components, packaging and manufacturing technology*, vol. 3, no. 11, pp. 1842–1850, 2013.

[71] B. Shi, A. Srivastava, and A. Bar-Cohen, "Co-design of microfluidic heat sink and thermal through-silicon-vias for cooling of three-dimensional integrated circuit," *IET Circuits, Devices & Systems*, vol. 7, no. 5, pp. 223–231, 2013.

[72] B. Shi and A. Srivastava, "Optimized micro-channel design for stacked 3-d-ics," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 1, pp. 90–100, 2013.

[73] D. Lorenzini, C. Green, T. E. Sarvey, X. Zhang, Y. Hu, A. G. Fedorov, M. S. Bakir, and Y. Joshi, "Embedded single phase microfluidic thermal management for non-uniform heating and hotspots using microgaps with variable pin fin clustering," *International Journal of Heat and Mass Transfer*, vol. 103, pp. 1359–1370, 2016.

[74] D. Lorenzini-Gutierrez and S. G. Kandlikar, "Variable fin density flow channels for effective cooling and mitigation of temperature nonuniformity in three-dimensional integrated circuits," *Journal of Electronic Packaging*, vol. 136, no. 2, p. 021007, 2014.

[75] H. Zhu, F. Hu, H. Zhou, D. Z. Pan, D. Zhou, and X. Zeng, "Interlayer cooling network design for high-performance 3d ics using channel patterning and pruning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 4, pp. 770–781, 2017.
[76] Y. Zhang, T. E. Sarvey, and M. S. Bakir, "Thermal challenges for heterogeneous 3d ics and opportunities for air gap thermal isolation," in *2014 International 3D Systems Integration Conference (3DIC)*. IEEE, 2014, pp. 1–5.